Lab 5: SSH Certificates, Authorisation, Hardening, and the Agent-Forwarding Hijack

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

SSH is the authentication surface you are most likely to administer in practice. This lab takes SSH beyond the password and the lone public key. You will stand up an **SSH certificate authority**, issue short-lived **user certificates**, and then do the part that most treatments skip: use the certificate's **principals** to drive an actual *authorisation* policy (who may become which account), and **revoke** an issued certificate to see the difference between expiry and revocation. You will then **harden** `sshd`, and finally execute and defend the **agent-forwarding hijack**, an attack on a convenience most SSH users switch on without thinking. Three machines take part: the VM, which is your workstation and runs the `ssh` client, and two containers inside it, a jump host and a target, each running `sshd`. No second computer is needed.

This lab is the whole of ”authentication *and* authorisation” in one artefact, and Part B is where you separate the four things a login depends on.

Contents

- Learning Objectives
- Before the lab (at home, ~30-45 min)
- Part A: Key-based authentication and its limits
- Part B: An SSH certificate authority: authentication, authorisation, revocation
- Part C: Hardening `sshd`
- Part D: The agent-forwarding hijack
- Optional take-home extension: host certificates
- If it does not work
- Reflection
- References
- Deliverables: Lab report 2 (individual)
- Appendix: the flags used in this lab

Learning Objectives

By completing this lab, students will be able to:

1. Configure key-based SSH authentication and state the two operational problems raw keys leave unsolved.
2. Operate an SSH certificate authority: sign user certificates and make a host trust the CA.
3. Enforce **authorisation** through certificate **principals** (`AuthorizedPrincipalsFile`), separating *what the CA authenticates* from *what local policy authorises*.
4. **Revoke** an issued certificate and articulate the difference between authentication, authorisation, expiry and revocation.
5. Harden an `sshd` configuration to a defensible baseline and justify the changes that matter most against a threat.
6. Execute the agent-forwarding hijack against a compromised jump host, and choose a correct mitigation.

What from this lab is assessed. This lab's write-up **is Lab report 2**, the one assessed deliverable of Block 2. It also collects the pieces Labs 3 and 4 asked you to keep: their preparation questions, Lab 4's flow sketch and 5c, and the reflections of all three labs. **Deliverables**, at the end of this handout, lists exactly what goes in.

Before the lab (at home, ~30-45 min)

Do all of this *before* the session. Starting the containers **pulls their images**, which needs network and time, and generating your keys in advance keeps the session for the security work. Arrive with the stack built and **both keypairs already generated**. If you shut the VM down in between, `docker compose up -d` in `~/ssc-lab5` brings the stack back.

Boot the VM and snapshot `lab5-clean`. Bring up the two-container topology (provided as `lab5-files.zip`: a `docker-compose.yml` defining `jump` and `target`, each running `sshd`, and `target` carrying the two unprivileged accounts the authorisation exercise uses, `ops` and `dev`):

Copy the zip into the VM first, from a terminal on your own computer, in the folder where you downloaded it:

```
scp -P 2222 lab5-files.zip student@localhost:
```

Then, in the VM (in your `ssh -p 2222 student@localhost` session):

```
mkdir -p ~/ssc-lab5 && cd ~/ssc-lab5
unzip ~/lab5-files.zip -d ~/ssc-lab5

# The CA first, because docker-compose.yml mounts its public half into both
# containers and it has to exist before they start. It lives in ~/.ssh/ with your
# other keys, and it is the ONE CA this lab uses: Exercise 2 signs with this same
# file. If you ever end up with two, the containers will trust one and your
# certificates will be signed by the other, and nothing will work. No passphrase,
# because this CA is disposable: a real CA key is the most protected key you own.
ssh-keygen -t ed25519 -f ~/.ssh/user_ca -N '' -C 'SSC user CA'

docker compose up -d
docker compose ps           # 'jump' and 'target' should be running
```

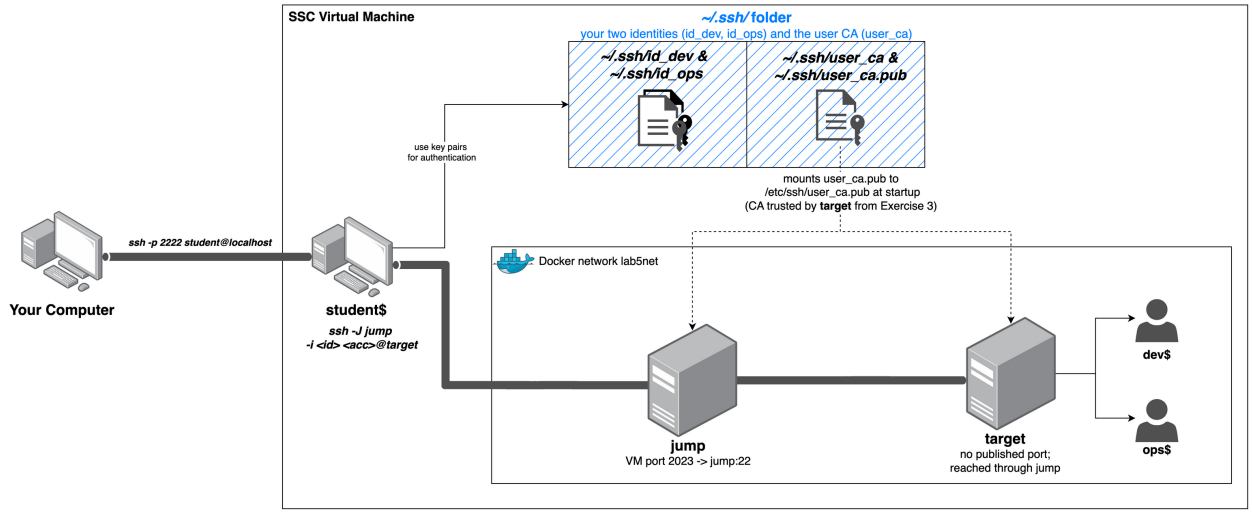
Generate **two separate keypairs**, one per identity you will hold. This matters mechanically: `ssh-keygen -s` writes its certificate next to the public key it signed, as `<name>-cert.pub`. Signing the *same* public key twice overwrites the first certificate, and later in this lab you need two certificates to exist **at the same time**.

```
ssh-keygen -t ed25519 -f ~/.ssh/id_dev -N '' -C 'lab5 dev identity'
ssh-keygen -t ed25519 -f ~/.ssh/id_ops -N '' -C 'lab5 ops identity'
```

What the shipped stack gives you. The `jump` and `target` containers (Alpine plus OpenSSH) build and run `sshd` as shipped, and `target` already carries the `ops` and `dev` accounts. Everything the exercises ask for works on that stack: certificates, an `AuthorizedPrincipalsFile`, a KRL, and agent forwarding from the VM through `jump` to `target`. Two container-specific differences are called out in the exercises below.

Throughout: **the VM is your workstation**, `jump` is a bastion you connect *through*, and `target` is the machine you ultimately reach.

The diagram shows where the pieces live and which command reaches which, not what each connection carries: that is what Parts B and D are for. `jump` and `target` are containers inside the VM, on one Docker network. Only `jump` has a port published on the VM; `target` has none, and its name resolves only inside that network, so you reach it through `jump`.



Two `sshd` commands you will use many times, always inside a container. `sshd -t` checks `sshd_config` for errors and prints nothing when it finds none. `sshd -T` prints the effective configuration `sshd` would run with if started now from the current file, one keyword per line (Lab 1 used it); a daemon you have not reloaded is still running the old one. After every edit: `sshd -t`, then reload with `kill -HUP 1`. The flags of `ssh`, `ssh-keygen` and `ssh-add` used in this lab are explained in the appendix.

Preparation questions (submit with Lab report 2, this lab's report)

- **P1.** Confirm `jump` and `target` are running (`docker compose ps`) and paste the output, together with `ls ~/.ssh/id_dev.pub ~/.ssh/id_ops.pub`.
- **P2.** In one sentence, name the two operational problems that plain SSH public keys leave unsolved (Part A explores them).

Part A: Key-based authentication and its limits

Exercise 1. Install the `dev` public key for the `dev` account on `target` (the stack README shows how, from the VM) and log in through `jump`:

```
# First: name the hosts and install your key. Both are in the stack README, under
# "Naming the two hosts" and "Installing your public key on target". Neither is
# optional and nothing resolves `jump` until you do the first.
ssh -J jump -i ~/.ssh/id_dev dev@target    # connect to target as dev, through the
↪  jump host
```

1a. With public-key authentication the server never receives a reusable secret from you. Two operational problems remain. How did `target` come to trust your public key, and how would that scale to hundreds of hosts and users? And on the first connection you were asked to accept two host key fingerprints, one for `jump` and one for `target`: what attack does blind acceptance expose you to, and why does it not scale either?

The second problem has a certificate answer too, host certificates, the mirror image of the user certificates you are about to issue. Lecture 5 covers the mechanism and the configuration (`ssh-keygen -h`, `HostCertificate`, `@cert-authority`), and the take-home extension at the end of this handout lets you build it. This session concentrates on the *user* side, where authorisation lives.

Part B: An SSH certificate authority: authentication, authorisation, revocation

A CA-signed certificate binds a key to one or more **principals** with an expiry, so hosts trust the **CA** once instead of every individual key. That closes the *authentication* gap of Part A. The principals then drive *authorisation*.

Exercise 2: Issue a user certificate.

You created the CA in the pre-lab, because the stack needs its public half at boot. It is the trust anchor: everything in Part B rests on it, and on nobody else holding it.

```
# sign the dev key with the principal 'dev', valid 1 hour, serial 1
ssh-keygen -s ~/.ssh/user_ca -I 'student@lab5' -n dev -z 1 -V +1h ~/.ssh/id_dev.pub
ssh-keygen -Lf ~/.ssh/id_dev-cert.pub        # inspect the issued certificate
```

2a. Read the certificate (–L): record its **principals**, **validity interval**, and **serial**. What does a *short* validity interval buy you compared with a permanent `authorized_keys` entry? Be precise about what it does **not** do.

⚠ **This certificate stops working in one hour.** From Exercise 3 onwards it is the **only** way into `target`, because Exercise 3 removes the raw `authorized_keys`. So if you finish this lab in a later sitting, or simply run long, the certificate will have expired and every connection to `target` will fail: with a password prompt before Exercise 6, and with `Permission denied (publickey)` after it. Neither says anything about expiry.

The fix is to issue a new one, with the command you just ran. Nothing else changes: the CA is the same, `target` already trusts it, and the principal policy already names you. Re-issuing a credential on expiry is the ordinary operation of a certificate authority. Compare that with what it would take to rotate a key you had distributed into `authorized_keys` on every host.

Exercise 3: Make `target` trust the user CA. On `target` (open a root shell on it from the VM with `docker compose exec target sh`), add the line `TrustedUserCAKeys /etc/ssh/user_ca.pub` to `/etc/ssh/sshd_config` (the stack mounts the user CA public key at that path); remove the raw key you installed in Exercise 1 (`/home/dev/.ssh/authorized_keys`); check with `sshd -t` and reload with `kill -HUP 1`. Then leave the container (`exit`) and, back on the VM, log in with the certificate:

```
ssh -J jump -i ~/.ssh/id_dev dev@target
```

If SSH asks for a password from here on, do not type one: press Ctrl-C. Until Exercise 6 the shipped `sshd_config` still accepts passwords, so a password prompt means certificate authentication did not succeed and the client has moved on to the next method it is allowed. The client is not told why, but `target` is. To read its reason, open a **second** terminal on your own computer, `ssh -p 2222 student@localhost` into the VM again, go to `~/ssc-lab5` and leave this running:

```
docker compose logs -f --tail 0 target    # only what target logs from now on;
↪ Ctrl-C to stop
```

Then repeat the attempt in your first terminal. If it reached `target`, two lines appear in the second: the server's reason, then `Connection closed by authenticating user <account>`. If nothing appears, the attempt failed before reaching `target`, and the client's own message is the one to read. (Reading the whole log afterwards is not the same: it keeps older attempts, and an old refusal for the same account looks exactly like a new one.)

3a. `target` now accepts you without ever having seen your individual key. Which accounts on `target` could this certificate log into right now? Try `ssh -J jump -i ~/.ssh/id_dev ops@target` without the log terminal and record what the client does, then find in the `sshd_config(5)` manual page (`man 5 sshd_config`, or online at https://man.openbsd.org/sshd_config) the rule `sshd` applies when no `AuthorizedPrincipalsFile` is set. Only after writing your answer, repeat the attempt with the log running in your second terminal, and compare its reason with yours. Exercise 4 replaces that implicit rule with a policy you write.

Exercise 4: Authorisation by principals. You will now make `target` map certificate principals to accounts with an explicit `AuthorizedPrincipalsFile`.

You added the first of these two lines in Exercise 3. **Neither is in the shipped `sshd_config`**: writing them is the exercise. On `target`, `sshd_config` should now read:

```
TrustedUserCAKeys /etc/ssh/user_ca.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
```

Add the second one now. Confirm from the VM, before and after:

```
docker compose exec -T target sshd -T | grep -i authorizedprincipalsfile
```

Before you add it the daemon prints `authorizedprincipalsfile none`: the default is not silence, it is a decision.

`%u` is the *login account* being requested. Each file lists the certificate principals allowed to become that account. Populate them so that the principal `ops` may log in as the `ops` account, and `dev` as the `dev` account:

```
# on target (docker compose exec target sh): you are root inside, and the image
# is alpine + openssh, so there is no sudo
mkdir -p /etc/ssh/auth_principals
echo ops > /etc/ssh/auth_principals/ops
echo dev > /etc/ssh/auth_principals/dev
kill -HUP 1        # reload sshd: it is PID 1 in the container, there is no systemd
exit               # back to the VM: the CA and your keys live there, not in target
```

Now, **on the VM**, issue a **second** certificate, for the `ops` identity, from the second keypair. Note that this writes `id_ops-cert.pub`, leaving `id_dev-cert.pub` intact: you now hold two live certificates.

```
ssh-keygen -s ~/.ssh/user_ca -I 'student-ops' -n ops -z 2 -V +1h ~/.ssh/id_ops.pub
ssh-keygen -Lf ~/.ssh/id_ops-cert.pub          # confirm: principal 'ops', serial 2
ls ~/.ssh/id_*-cert.pub                        # confirm: BOTH certificates exist
```

Two things that differ in a container. (1) Reload `sshd` with `kill -HUP 1`, not `systemctl reload ssh`: the container runs `sshd -D` as PID 1 and has no systemd. (2) `AuthorizedPrincipalsFile` must have safe ownership and modes, or `sshd` refuses it with "bad ownership or modes"; root-owned is the simple case. Because you run these inside `target` as root (`docker compose exec target ...`), the files you create are root-owned automatically. Do not bind-mount them from the VM: the host UID maps to a non-root user inside and `sshd` will reject them.

4a. With the **dev** certificate, try to log in as `ops`, then as `dev`:

```
ssh -J jump -i ~/.ssh/id_dev ops@target
ssh -J jump -i ~/.ssh/id_dev dev@target
```

Record what each attempt does, and for any that does not get in, the reason that appears in your log terminal. Both present the same CA-signed credential: explain the outcome in terms of the principal-to-account mapping.

4b. Now confirm the mapping in the other direction with the **ops** certificate (`ssh -J jump -i ~/.ssh/id_ops ops@target`). Then state, in one sentence each: what the **CA** decided, and what the **`AuthorizedPrincipalsFile`** decided. Two different decisions, in two different places, under two different owners.

Exercise 5: Revocation. Suppose the `ops` certificate leaks and you must stop it working *now*, before it expires. Build a Key Revocation List (KRL) and have `target` honour it, in two steps.

First, **on the VM**, revoke the `ops` certificate into a KRL, check it, and copy it into `target`:

```
ssh-keygen -k -f ~/revoked_keys ~/.ssh/id_ops-cert.pub     # -u to update an
 ↪  existing KRL
ssh-keygen -Qf ~/revoked_keys ~/.ssh/id_ops-cert.pub       # what does it report?
cd ~/ssc-lab5
```

```
docker cp ~/revoked_keys target:/etc/ssh/revoked_keys
```

Copying the file is not enough: `sshd` ignores a KRL until its configuration names it. So, second, **on `target`** (`docker compose exec target sh`), add the line `RevokedKeys /etc/ssh/revoked_keys` to `/etc/ssh/sshd_config`, check with `sshd -t`, reload with `kill -HUP 1`, and `exit` back to the VM.

5a. Try both certificates again and record both outcomes, with the reason that appears in your log terminal for any that does not get in.

```
ssh -J jump -i ~/.ssh/id_ops ops@target
ssh -J jump -i ~/.ssh/id_dev dev@target
```

5b. **The four-way distinction (the point of Part B).** In one sentence each, say what each of these *terminates or bounds*, and how they differ:

- the **CA signature**;
- the **`AuthorizedPrincipalsFile`**;
- the **validity interval**;
- the **KRL / `RevokedKeys`**.

Why can you not achieve 5a's effect with short lifetimes alone?

Part C: Hardening `sshd`

Exercise 6. On `target` (`docker compose exec target sh`), append these five hardening directives to the `sshd_config` you have been editing (your Part B lines stay where they are):

```
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
AuthenticationMethods publickey
```

Check with `sshd -t`, reload with `kill -HUP 1`, `exit` back to the VM, and verify the effect from there:

```
ssh -o PreferredAuthentications=password -J jump dev@target   # must now fail
ssh -J jump -i ~/.ssh/id_dev dev@target                        # must still succeed
 ↪ (certificate)
```

Now read the effective configuration, which is not the file. Lab 1 taught you the instrument; this is where you need it:

```
docker compose exec -T target sshd -T | grep -iE \
  -e '^passwordauthentication|^permitrootlogin|^kbdinteractiveauthentication' \
  -e '^pubkeyauthentication|^authenticationmethods'
```

6a. **Do the two checks agree?** Paste the `sshd -T` output into your report, then compare each of the five values with the value you meant to set. Answer three counts, because they can be different numbers. For your five directives:

- how many of the **five settings you wanted** are in force;
- how many of the **lines you wrote or edited** are the ones the daemon actually obeys;
- how many of the **lines you wrote or edited** are **inert**.

Read `target`'s `sshd_config` from the top before you answer, and note that `sshd -t` will not help you: it reported no error. If any of your lines is inert, say **why**, then make your settings take effect, reload, and show `sshd -T` reporting the five values you wanted. If none is, say for each of the five settings where it is in the file and whether you edited, added or left that line. Then predict what `sshd -T` would have shown had you appended the five lines at the end, as Exercise 6 asked, and justify the prediction from the `sshd_config(5)` manual page.

Finally, say what this means for the verification you ran a moment ago. The password attempt's error message lists the methods the server still offers: **which single directive removed password from that list**, and why could passing that test not prove that the rest of the baseline was applied?

6b. Pick the **three** directives above that most reduce this host's remote attack surface. For each, name in one sentence the threat it addresses. Then say which single one you would apply first if you could only make one change, and why. (Do not write a line for every directive: the point is the ranking, not the inventory.)

6c. **A server left on the default.** Many servers you will meet run with `PasswordAuthentication` left **on**. Given what you configured here, state in one sentence why that single default re-opens an initial-access *path* that the hardened config had closed.

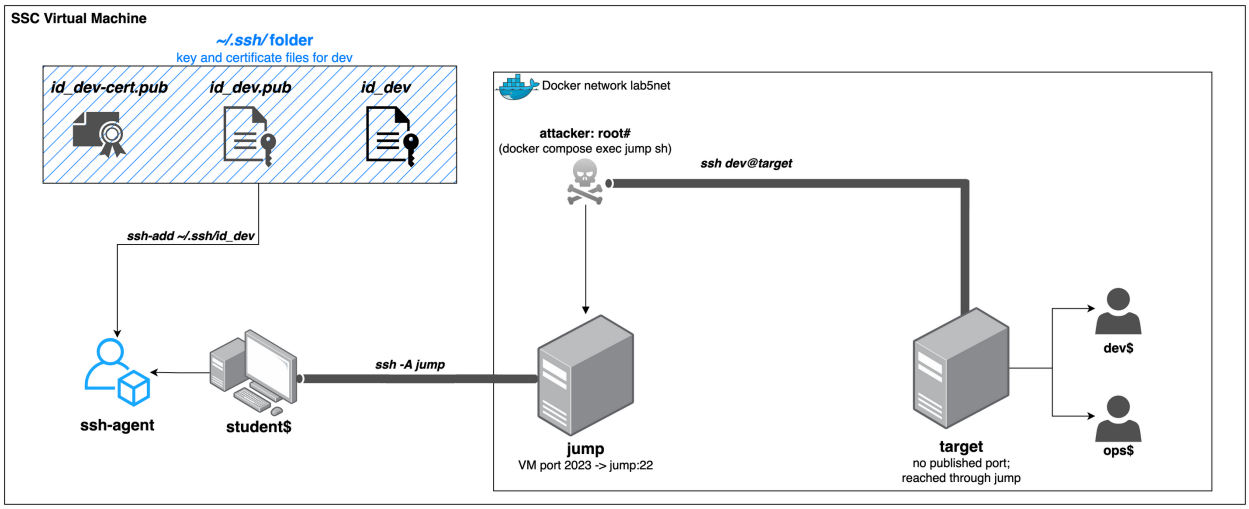
Part D: The agent-forwarding hijack

An **SSH agent** (`ssh-agent`) is a helper program on your workstation into which you load keys with `ssh-add`, so that `ssh` can use them without reading the key file, or asking for its passphrase, every time. Programs on the same machine reach the agent through a Unix socket whose path is in the environment variable `SSH_AUTH_SOCK`. Agent forwarding (`ssh -A`) lets a remote host use your *local* SSH agent to authenticate onward, which is convenient for chaining through a bastion. This part shows what it costs.

Exercise 7: Execute the hijack.

If you are starting Part D in a later sitting, re-issue your dev certificate first (the Exercise 2 command, unchanged). The hijack ends by authenticating to `target`, and `target` has trusted nothing but the CA since Exercise 3. Check with `ssh-keygen -Lf ~/.ssh/id_dev-cert.pub`. **Do this before loading the key into the agent**: `ssh-add` picks up <key>`-cert.pub` only if it exists at the time you add the key, so an agent loaded first holds the key without the certificate.

Part D uses two shell sessions on the VM, each in its own terminal on your computer: one plays the user who forwards their agent, the other the attacker who controls `jump`.



Connect to `jump` **with agent forwarding**, as a careless user would:

```
eval "$(ssh-agent -s)"          # ensure an agent is running in this shell
ssh-add ~/.ssh/id_dev           # load your key into the agent
ssh -A jump                     # forward the agent to the (compromised) jump host
```

Leave that session open: the forwarded socket exists only while it lasts. The `jump` container is playing the role of a **compromised bastion**. For the attacker's side, open a **second** terminal on your own computer, `ssh -p 2222 student@localhost` into the VM again, and in `~/ssc-lab5` take a root shell on `jump` with `docker compose exec jump sh` (Docker's control of the container stands in for the compromise). There you do three things, in order, and record what each one gives you:

1. **Look for the victim's private key on this host.** The `find` searches the whole filesystem for any file whose name starts with `id_dev`; `2>/dev/null` discards the errors from places it cannot read.
2. **Look for a forwarded agent socket** and keep its path in the shell variable `SOCK`. Where it would be depends on the OpenSSH version, so the commands try two places (see the note after them).
3. **Run the last two commands and record what each prints.** `SSH_AUTH_SOCK=$SOCK command` sets the variable for that one command only.

```
# on jump, as root: first look for the private key anywhere on this host
find / -name 'id_dev*' -type f 2>/dev/null              # what is here?
# the forwarded agent socket (path is OpenSSH-version-dependent, see the note):
SOCK=$(ls -t /home/dev/.ssh/agent/s.* 2>/dev/null | head -1)      # modern OpenSSH,
 ↪  newest first
[ -z "$SOCK" ] && SOCK=$(ls -t /tmp/ssh-*/agent.* 2>/dev/null | head -1)   # older
 ↪  OpenSSH
echo "$SOCK"
SSH_AUTH_SOCK=$SOCK ssh-add -L                          # what does this list?
SSH_AUTH_SOCK=$SOCK ssh dev@target                      # and who are you now?
```

Where the forwarded socket lives. Older OpenSSH placed the forwarded agent socket under `/tmp/ssh-*/agent.*`; the version on the course image places it under the user's home, `~/.ssh/agent/s.*`. The command above tries both.

7a. Explain exactly what the attacker used to reach `target`, and why keeping the private key secret did not protect you here.

Exercise 8: Mitigate. Close the `ssh -A` session and leave the root shell on `jump`. Then, from a shell on the VM, reproduce the connection **without** exposing yourself:

```
ssh -J jump -i ~/.ssh/id_dev dev@target
```

8a. Explain why `ProxyJump` (`-J`) is not vulnerable to the Part D hijack when agent forwarding is. State the rule you would give an operator: when, if ever, is `-A` acceptable, and what is the safe default? Mention `ssh-add -c` (confirmation) and per-host `ForwardAgent no` as defence-in-depth.

When you finish, run `docker compose down` in `~/ssc-lab5`, and keep your keys and certificates until the report is written. Restoring the `lab5-clean` snapshot afterwards gives you a clean VM.

Optional take-home extension: host certificates

Not required, not marked, and not part of Lab report 2. If you want to close the trust-on-first-use gap from Part A yourself, sign `target`'s host key with a **host** CA. The `-h` flag is what makes it a *host* certificate; without it you get a user certificate, which `HostCertificate` cannot load.

```
# on the VM, in ~/ssc-lab5
ssh-keygen -t ed25519 -f ~/.ssh/host_ca -N '' -C 'SSC host CA'
docker cp target:/etc/ssh/ssh_host_ed25519_key.pub target_host_ed25519.pub   #
 ↪  target's host key
ssh-keygen -s ~/.ssh/host_ca -I target -h -n target -V +1h target_host_ed25519.pub
docker cp target_host_ed25519-cert.pub
 ↪  target:/etc/ssh/ssh_host_ed25519_key-cert.pub
# inside target: add 'HostCertificate /etc/ssh/ssh_host_ed25519_key-cert.pub' to
 ↪  its sshd_config,
#   then sshd -t && kill -HUP 1   (PID 1; no systemd in the container)
# back on the VM, trust the host CA for the name target:
echo "@cert-authority target $(cat ~/.ssh/host_ca.pub)" >> ~/.ssh/known_hosts
```

Reconnect: you should no longer be prompted to accept `target`'s host key (`jump`'s is still a plain key). What does your client now verify, and how does that close the man-in-the-middle window?

If it does not work

The first two look like your certificate is wrong when it is not.

What you see	What to do
A password prompt on `dev@target` (or, after Exercise 6, `Permission denied (publickey)`), after everything worked earlier	**Your certificate has probably expired.** It was issued for one hour in Exercise 2, and from Exercise 3 it is the only way in. Check its validity with `ssh-keygen -Lf ~/.ssh/id_dev-cert.pub`. If it has expired, re-issue it with the Exercise 2 command; if the interval is still valid, repeat the attempt with the log running in a second terminal (the note before 3a) for the server's reason. The client names none of this, which is itself worth noticing: an expired credential and a wrong one fail identically on your side.
The hardened directives in Exercise 6 have no effect, and `sshd -t` says nothing	The directives are written but not in force. See 6a: read `sshd -T`, not the file, and find out why.
A password prompt (or `Permission denied`), and repeating the attempt with the log terminal running (the note before 3a) shows `Certificate does not contain an authorized principal`	The certificate is valid; the account will not accept that principal. Check the principal you signed with against the contents of `/etc/ssh/auth_principals/<account>`, then check that file's ownership.
sshd logs `bad ownership or modes for .../auth_principals/...`	The principals file is not root-owned. Create it **inside** `target` with `docker compose exec target ...`, never bind-mounted from the VM: your host UID maps to a non-root user inside and `sshd` refuses the file.
`WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!` after `docker compose down` and `up`	`down` removes the containers, and each new container makes its own host keys when it starts, so your `known_hosts` no longer matches. Your `sshd_config` edits went with the old containers too. Remove the two old entries with `ssh-keygen -R '[127.0.0.1]:2023'` and `ssh-keygen -R target`, reinstall your public key on **both** `jump` and `target` (the two commands in the stack README), then redo the exercises you had done on `target`. If you did the optional host-certificate extension, redo it too.
`systemctl reload ssh` fails inside `jump` or `target`	There is no systemd in the container; `sshd -D` is PID 1. Use `kill -HUP 1`, or `docker compose exec target kill -HUP 1` from the VM.
The hijack finds nothing under `/tmp/ssh-*/agent.*`	On this OpenSSH the forwarded socket is at `~/.ssh/agent/s.*`. Look there first; the `/tmp` path is the older-version fallback.

If none of these matches, read the client's message in full, then repeat the attempt with `target`'s log running (the note before 3a): together they usually name the layer that refused, and that is most of the answer. Then ask your lab teacher, in the session or by email.

Reflection

Your reflection goes into Lab report 2, as an appendix, beside the reflections from the other labs in this block. They are assessed with the report: every one has to be there, each answering its questions with a position you argue from what you did in the lab.

In a short piece (200 to 300 words):

1. Certificates replaced ”every host trusts every key” with ”every host trusts one CA”. That is the same move federated identity made in Lab 4 (every application trusts one identity provider). Compare the two: what does each centralise, and what becomes the new single point of failure?

2. Part B forced apart four things a login actually depends on: authentication (the CA), authorisation (the principal policy), expiry, and revocation. Pick a real incident (a leaked credential, a departing employee) and say which of the four is the *right* response, and why the other three would not suffice.
3. The agent-forwarding hijack needed no cryptographic weakness and no stolen key, only a convenient default and a compromised middle host. How does this echo the container-socket escape you will meet later, and the world-writable cron you found in Lab 1?

References

- `ssh-keygen(1)`: certificate options (`-s`, `-I`, `-n`, `-z`, `-V`, `-L`, `-k`, `-Q`).
- `sshd_config(5)`: `TrustedUserCAKeys`, `AuthorizedPrincipalsFile`, `RevokedKeys`, `AuthenticationMethods`, `HostCertificate`.
- OpenSSH certificate format: `PROTOCOL.certkeys` (principals, critical options, extensions), https://github.com/openssh/openssh-portable/blob/V_10_0_P1/PROTOCOL.certkeys. ⚠ Pinned to a tag: the file was **removed upstream after OpenSSH 10.0** and is not on `master`. For KRLs see `PROTOCOL.krl`, which is current, and `ssh-keygen(1)`.
- `ssh-agent(1)` and `ssh_config(5)`: `ForwardAgent`, `ProxyJump`; `ssh-add -c`.
- Facebook Engineering, "Scalable and secure access with SSH" (SSH CA and principals at scale).
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022: the PKI/authentication chapters.

Deliverables: Lab report 2 (individual)

Lab report 2 is the write-up of this lab, submitted **individually** though the work is done in your group of three. It is also where **Block 2's preparation questions are collected**: include P1-P2 from Lab 3, P1-P3 from Lab 4, and P1-P2 from this lab, with the proof-of-setup output each asks for, and from Lab 4 also your annotated flow sketch (Exercise 1a) and your answer to 5c. They are required. The same goes for **the reflections from Labs 3, 4 and this one**, which go in as an appendix and are assessed with the report.

Beyond those, the report comprises:

1. The two operational problems raw keys leave open, and how certificates solve the first (Parts A and B).
2. Your inspected user certificate: principals, validity, serial (Exercise 2).
3. The **authorisation** result: which principal became which account and why, with the two-decisions answer (Exercise 4).
4. The **revocation** result and the four-way distinction (Exercise 5).
5. Part C: the `sshd -T` output, the three counts, why any inert lines are inert and the fix with `sshd -T` agreeing (or, if none was, where you put your lines and your justified prediction), and what the password test could and could not prove (6a); your **three** ranked directives with a threat each (6b); and your answer to 6c.
6. The agent-forwarding hijack: what the attacker used, and your chosen mitigation with the operator rule (Part D).

Include the commands you ran, the `ssh-keygen -L` output of both certificates, and `target`'s final `sshd_config`. The optional host-certificate extension is **not** marked.

Marked on a compact rubric (correct CA operation; a correct authorisation model and revocation; soundness of the hardening ranking; a correct account of the hijack and its mitigation), plus the block's reflections. Due on Blackboard by **Sunday 1 November**, one week after Block 2 ends; the exact time is the one on the Blackboard assignment, and the late-report rule applies (one late report is free, up to seven days; see the assessment timeline).

Appendix: the flags used in this lab

Every flag below is used or mentioned in this handout. The manual pages (`man ssh-keygen`, `man ssh`, `man ssh-add`) have the full list.

`ssh-keygen`

Flag	What it does here
`-t ed25519`	The type of key to generate.
`-f <file>`	The file to work on: the key to create, the certificate to read (`-Lf`), or the KRL to create or query (`-k`, `-Qf`).
`-N ''`	The passphrase for the new key; `''` means none, acceptable only for disposable lab keys.
`-C <text>`	A comment stored with the key, to tell keys apart.
`-s <ca key>`	Sign a public key with this CA private key, producing `<name>-cert.pub` next to it.
`-I <id>`	The certificate's key identity. `sshd` writes it to its log when the certificate is used.
`-n <principals>`	The principals written into the certificate, comma-separated; here one each, `dev` or `ops`.
`-z <serial>`	The certificate's serial number.
`-V <interval>`	The validity interval; `+1h` means from now until one hour from now.
`-h`	With `-s`: create a **host** certificate instead of a user certificate (optional extension).
`-L`	Print the contents of a certificate (with `-f`, as `-Lf`).
`-k`	Generate a Key Revocation List (KRL) from the keys or certificates given.
`-u`	With `-k`: add to an existing KRL instead of replacing it.
`-Q`	Test whether the keys or certificates given are revoked by a KRL (with `-f`, as `-Qf`).
`-R <host>`	Remove a host's entries from your `known_hosts` file.

`ssh`, `scp`, `ssh-add` and `ssh-agent`

Command and flag	What it does here
`ssh -p <port>`	Connect to this port (`2222` reaches the VM from your computer). `scp` uses a capital `-P` for the same thing.
`ssh -i <key>`	The private key to authenticate with. If `<key>-cert.pub` exists next to it, the certificate is offered too.
`ssh -J jump`	Reach the destination through `jump` (`ProxyJump`).
`ssh -A`	Forward your agent to the host you log into (Part D).
`ssh -o <Option>=<value>`	Set one configuration option for this connection, as in `-o PreferredAuthentications=password`.
`ssh-add <key>`	Load a private key into the agent, with its certificate if `<key>-cert.pub` exists at that moment.
`ssh-add -L`	List the public keys and certificates the agent holds.
`ssh-add -c`	Load a key so that every use asks you for confirmation (Exercise 8a).
`ssh-agent -s`	Start an agent and print the shell commands that set `SSH_AUTH_SOCK`; `eval` runs them.